```
UPDATE "sqlite_sequence" SET "seq" = 0 WHERE "name" IN
    ('bands_band_musicians', 'django_session', 'bands_room',
    'content_seekingad', 'auth_group_permissions',
    'auth_user_user_permissions', 'bands_musician', 'auth_group',
    'auth_user_groups', 'bands_userprofile',
    'bands_userprofile_venues_controlled', 'bands_venue',
    'django_admin_log', 'bands_userprofile_musician_profiles',
    'auth_user', 'auth_permission', 'bands_band',
    'django_content_type');
COMMIT;
```

As you might expect from `flush`, all rows from almost all the tables get deleted. The ID key also gets reset to start over again, hence the `SET "seq" = 0` part of the `UPDATE` statement.

The `sqlmigrate` command is a little more complex. It requires the name of an app and the name of the migration in question. To see the first change made to the bands app after it was initially created, you specify `bands` and `0002` (you only need to provide enough of the migration file name for it to be uniquely found—you don't have to type the whole thing):

```
(venv) RiffMates$ python manage.py sqlmigrate bands 0002
BEGIN;
--
-- Create model Venue
--
CREATE TABLE "bands_venue" ("id" integer NOT NULL PRIMARY KEY
    AUTOINCREMENT, "name" varchar(20) NOT NULL);
--
-- Create model Room
--
CREATE TABLE "bands_room" ("id" integer NOT NULL PRIMARY KEY
    AUTOINCREMENT, "name" varchar(20) NOT NULL, "venue_id"
    bigint NOT NULL REFERENCES "bands_venue" ("id")
    DEFERRABLE INITIALLY DEFERRED);
CREATE INDEX "bands_room_venue_id_16903c8e" ON "bands_room" ("venue_id");
COMMIT;
```

The result, in this case, is the SQL that got run to create the tables for the `Venue` and `Room` `Models` as well as the database index on the column that relates the two. As everything you do to your models gets done through the migration mechanism, you can always see the underlying SQL with this command.

9.2 Writing your own management command

When you run manage.py on its own and get the list of all the installed management commands, the result is grouped by Django app. As you might guess from this, you can write your own management command. This can be useful to interact with your project data from the command line, for example running batch jobs or doing the occasional bit of clean-up.